

PART 12 OF 18

AI Observability Architecture

Reasoning Telemetry, Decision Audit Trails, Context Lineage, Behavioral Analytics, OpenTelemetry for AI, Arize Phoenix, Langfuse, and Trust Scoring

ENTERPRISE AI CONTROL ARCHITECTURE

Implementation Guide for Production AI Systems • 2026

12.1 Beyond Traditional Observability: The AI Observability Stack

Traditional observability (logs, metrics, traces) captures what software systems did in sufficient detail for operations and debugging. For AI agents, this is necessary but insufficient: logs tell you what API calls were made; AI observability additionally captures why those calls were made, what the agent was thinking, and whether the reasoning process was sound. This semantic dimension of observability is unique to AI systems and requires new instrumentation primitives.

12.1.1 AI Observability Dimensions

Execution Observability

What: tool calls, API requests, outputs. Standard distributed tracing handles this layer. **Implementation:** OpenTelemetry traces

Context Observability

What information was in the agent's context when each decision was made. Essential for debugging and audit. **Implementation:** Context snapshots in trace spans

Reasoning Observability

Why the agent made each decision. Includes chain-of-thought, goal state, planning traces. **Implementation:** LLM-specific span attributes

Memory Observability

What memories were retrieved and written; how memory influenced decisions. **Implementation:** Memory read/write events in trace

Goal Observability

How the agent's goal state evolved throughout the task; goal drift detection. **Implementation:** Goal state events timeline

Trust Observability

Real-time trust scores; policy decisions; authorization events. **Implementation:** Policy decision events

Behavioral Observability

Statistical behavioral patterns; drift from baseline; anomaly detection. **Implementation:** Behavioral analytics platform

12.2 OpenTelemetry for AI Agents

12.2.1 OpenInference Semantic Conventions

OpenInference (Arize AI's contribution to OpenTelemetry for AI) defines semantic conventions for LLM and agent observability. These conventions standardize how AI systems instrument their traces, enabling consistent analysis across different frameworks and vendors.

OTel Attribute	Content	Security Use
gen_ai.prompt	Input prompt to LLM call	Injection detection; prompt audit
gen_ai.completion	LLM output text	Output safety scanning; PII detection
gen_ai.tool.name	Name of tool invoked	Tool usage audit; policy verification
gen_ai.tool.input	Tool invocation parameters	Parameter validation; DLP
gen_ai.tool.output	Tool result returned to model	Result injection detection; content scanning
llm.token.count.total	Token usage for the LLM call	Cost control; DoS detection
gen_ai.agent.id	Unique agent instance identifier	Agent identity correlation
gen_ai.agent.goal	Current agent goal state	Goal drift detection; intent verification
gen_ai.retrieval.query	Query sent to vector database	Retrieval monitoring; data access audit
gen_ai.retrieval.results	Documents retrieved	Knowledge base access audit

12.3 AI Observability Platform Comparison

Platform	Strengths	Weaknesses	Best For
Arize Phoenix	LLM eval framework; strong prompt/response analysis; open source; OTel integration	Limited multi-agent support; no enterprise RBAC in OSS version	LLM quality monitoring; prompt optimization
Langfuse	Excellent trace visualization; cost tracking; datasets and evals; strong OSS community	Newer platform; multi-agent tracing improving	Developer debugging; evaluation pipelines
Weights & Biases (Weave)	Strong experiment tracking; model versioning; good LLM support	ML engineering focus; less security-oriented	Model development and evaluation
Helicone	Low-latency proxy; cost monitoring; request logging	Limited semantic analysis; more operational than analytical	Cost management; request logging

Platform	Strengths	Weaknesses	Best For
Custom OpenTelemetry	Full control; integrate with existing Grafana/Datadog; compliance-ready	Significant engineering effort; no AI-specific dashboards out of box	Enterprises with existing OTEL investment
Datadog LLM Observability	Integrated with existing enterprise Datadog; good security team adoption	Limited LLM-specific depth vs dedicated platforms; cost	Enterprises already using Datadog

12.4 Reasoning Telemetry Architecture

12.4.1 Capturing Agent Reasoning Traces

Reasoning telemetry captures the agent's chain-of-thought, planning steps, and decision rationale in a structured, queryable format. This data is essential for security audit, compliance, debugging, and behavioral analytics. However, reasoning traces may contain sensitive business information and must be protected with the same care as the outputs.

Reasoning Trace Data Structure

■ Reasoning Trace Schema (JSON): { trace_id: 'tr-8f3a2c...', agent_id: 'finance-analyst-sess-a7f2b1', task_id: 'task-8c3d9e', step_number: 3, goal_state: 'Analyze Q2 revenue by region', reasoning: "", // Stored encrypted reasoning_summary: 'Decided to retrieve revenue data from data warehouse', planned_action: {tool: 'query_data_warehouse', parameters: {query_id: 'rev_by_region_q2'}}, risk_score: 15, policy_decision: 'APPROVE', timestamp: '2026-06-25T14:23:01Z' }

12.4.2 Reasoning Trace Privacy

- **Encryption at Rest:** Reasoning traces encrypted with agent-instance-specific keys; key rotation on session end
- **Access Control:** Reasoning traces accessible only to: the deploying team, SOC with audit justification, compliance under court order
- **Redaction Pipeline:** Automated PII detection and redaction in reasoning summaries before they enter analytics pipelines
- **Selective Disclosure:** Regulatory reporting receives summaries, not full traces; full traces available only through privileged access process

12.5 Behavioral Analytics for AI Security

Behavioral analytics applies statistical and ML methods to agent telemetry to detect anomalies, drift, and potential security incidents that rule-based systems cannot catch. The key insight is that even individually authorized actions can be anomalous in aggregate—detecting this requires statistical modeling of normal behavior.

Analytic	Method	Signal	Alert Threshold
Action frequency analysis	Z-score vs. baseline distribution	Sudden burst of specific action type	> 3 sigma from hourly baseline
Data access pattern	Graph-based access pattern comparison	Novel resource access or graph traversal	New node accessed in access graph
API endpoint diversity	Shannon entropy of API calls	Systematic API enumeration (low entropy)	< 2.0 bits entropy over 100 calls
External call correlation	Cross-agent external call analysis	Multiple agents calling same unusual endpoint	> 3 agents calling same new endpoint
Session risk trajectory	Time-series analysis of cumulative risk score	Accelerating risk accumulation	Risk score growth > 5 points/minute
Goal drift	Semantic similarity of actions to stated goal over time	Actions diverging from stated goal	Semantic similarity < 0.6
Credential access timing	Time-of-day and sequence analysis	Unusual credential access patterns	Access outside normal operational hours